

MessageFoundry — System Requirements

Early Access · MessageFoundry v0.3.2 · September 2026

This guide lists minimum and recommended requirements for the MessageFoundry (MEFOR) engine, message store, and administration clients. The engine runs as a headless Python/asyncio service. It serves the browser console at `/ui`.

Throughput references. `benchmarks/TUNING-BASELINE.md` is the canonical source for measured rates. `THROUGHPUT.md` explains how to read them. The `sizing tiers` derive from named measured runs. Daily figures apply a stated duty cycle rather than multiplying by `86,400`. Each tier inherits its source run's hardware, fan-out, and transform cost. Durable writes make throughput hardware-dependent. These figures are not guarantees. Establish a baseline on production-like hardware before go-live. Use `LOAD-TESTING.md`, `harness/load/`, and `Capacity notes`.

Hardware

	Minimum (lab / low-volume pilot)	Recommended (single-node production)
CPU	2 cores	4+ cores (transform throughput is per-core, one worker set per connection)
Memory	4 GB	8–16 GB
Disk	10 GB free, any disk	SSD, 50+ GB or sized to your retention window, on a low-latency local volume
Store volume	—	Put the message store on a fast local disk (not a network share). Budget for store + WAL growth : the staged pipeline writes ~3× per message on the embedded store (see <code>write-amplification benchmark</code>).

The engine and store may share a host for a low-volume pilot. For production, run a **server database** (PostgreSQL or SQL Server) on its own host, sized by your DBA, and keep the engine host dedicated. For volume beyond one CPU core, see `Sizing by message volume`.

Hardware memory encryption — required for an ASVS Level 3 PHI deployment

Requirement. A deployment claiming ASVS **Level 3** for PHI **must** run the engine on a host that provides **full memory encryption** — **AMD SEV-SNP** (EPYC 7003 "Milan" or later) or **Intel TDX** (5th Gen Xeon Scalable or later) — *and* must actually launch the engine's VM as a **confidential guest** on that platform. Capable silicon that is not running the guest in confidential mode does not satisfy it. `ASVS 11.7.1` states: *full memory encryption ...*

protects sensitive data while it is in use. PHI remains **plaintext in process memory** during parsing, routing, and transformation. HL7 uses `str` throughout processing. Application-level controls cannot encrypt the interpreter's heap.

The host must provide memory encryption. MessageFoundry cannot supply it. On a host without this protection:

- the engine **still runs** — nothing here is a functional requirement, and every other PHI control (at-rest encryption, retention, audit, RBAC, transport) is unaffected.
- ASVS 11.7.1 is capped at **Partial**, not Pass, and that cap is a **hardware fact about your deployment**, not a gap in the software. Disclose it in your own assessment rather than working around it.
- an **exposed** PHI instance **warns at every start** until the decision is recorded —
`[security].memory_encryption_operator_declared = true` is the operator's declaration that the host provides it. The engine **starts either way**. Memory encryption is a host property, and Windows operators cannot currently provide it on-premises. A default startup refusal would therefore block those deployments. An estate that has standardized on confidential-computing hosts can make the missing declaration fatal with `[security].require_memory_encryption_declaration = true` (default `false`). Loopback and synthetic instances are **silent but not exempt**. Exposure controls the warning to reduce unattended startup output. The memory risk still applies. `GET /security/posture` reports the property for every instance. See `CONFIGURATION.md` `[security]` and `OFF-LOOPBACK-DEPLOYMENT.md`.

The posture endpoint reports host information. `GET /security/posture` has these report-only fields: `memory_encryption_self_reported_capability`, `..._self_reported_active`, `..._self_reported_mechanism`, and `memory_encryption_readout_source`. Linux capability comes from `/proc/cpuinfo`. Linux activation comes from `/dev/sev-guest` or `/dev/tdx_guest`. On **Windows, all fields are null**.

These values do not satisfy 11.7.1. The host reports them about itself, and the host can be the adversary. Evidence requires a CPU-signed attestation report verified against the silicon vendor's root PKI. **That verification is not built.** The response includes this limit in `memory_encryption_note`. The deployment determines the assessment result.

Platform availability, verified 2026-07-22. Windows is the primary supported engine platform. On-premises Windows deployments currently cannot meet the memory-encryption requirement because their hypervisor does not provide the required confidential guest:

Platform	Confidential guest for a engine VM
Hyper-V on-premises (Windows Server 2019–2025)	❌ None. Windows Server 2025 ships no confidential VM. The vNext Insider "Trusted Launch" (Secure Boot + vTPM) is not memory encryption.
VMware ESXi 9.0	⚠️ SEV-SNP is a <i>Limited Availability</i> release and its guest requirements are stated in Linux-kernel terms. Windows is not listed as a supported SEV-SNP guest.
Azure / Azure Local confidential VMs	✅ SEV-SNP and TDX with Windows Server guests (a Microsoft paravisor supplies what a Windows guest needs).

Platform	Confidential guest for a engine VM
Linux guests on KVM / AWS / GCP / ESXi 9	✓ Where the host is SEV-SNP/TDX-capable and the guest is launched confidential.

As of the verification date above, Windows on-premises deployments are capped at **Partial**. Linux or Azure confidential VMs provide a route to the hardware property. Plan hardware purchases accordingly: SEV-SNP needs EPYC 7003+, and TDX needs 5th Gen Xeon Scalable+. These may be newer than servers on a 5–7-year hospital refresh cycle.

Operating systems

Platform	Status
Windows Server 2022 / 2025	✓ Primary supported & serviced platform (Windows-service deployment via NSSM)
Windows Server 2019	✓ Supported
Windows 10 / 11	✓ Supported (development, pilot, test-harness host)
Linux (modern x86-64 distributions)	✓ Engine supported (cross-platform Python). No bundled service installer — run under systemd yourself
macOS	⚠ Development / test-harness use only

Runtime

Component	Requirement
Python	3.14 , 64-bit (the only supported runtime, CI-validated on Linux + Windows Server 2022 + 2025, the primary deploy target)
Service manager (Windows)	NSSM (auto-provisioned, SHA-256-pinned, by the installer, or pre-staged). Requires administrator / elevation to register the service.
C compiler	Not required for the default install (runtime dependencies ship as wheels)

Databases (message store)

Database	Status	Driver / prerequisite
SQLite (WAL)	✓ Default, bundled — single-node	None (<code>aiosqlite</code> , in-process)
PostgreSQL 13+	✓ Production	<code>messagefoundry[postgres]</code> extra (<code>asyncpg</code> — no OS dependency. Ships compiled wheels)

Database	Status	Driver / prerequisite
Microsoft SQL Server 2022 / 2025	✓ Production	<code>messagefoundry[sqlserver]</code> extra (<code>aiodbc</code>) plus the OS-level Microsoft ODBC Driver 18 for SQL Server (18.5+ covers both majors). Read-Committed Snapshot Isolation (RCSI) recommended. SQL Server 2025 requires an AVX-capable CPU.
MySQL / Oracle	✗ Not supported (roadmap)	—

The embedded SQLite store needs no setup and suits pilots and single-node deployments. A **server database is greenfield-only** — there is no in-place migration from a populated SQLite store. Drain and cut over. The DB tier owns its own backup, HA, and (SQL Server) TDE / purge maintenance. A server database is also the **concurrency / scale substrate** — see below.

Administration clients

Client	Requirement
Web console (the operator UI)	The operator needs a modern browser, with no local installation. The engine serves <code>/ui</code> from its FastAPI app (ADR 0065). The console is on by default for loopback binds (ADR 0143). Set <code>[security].serve_web_console = false</code> for a JSON-API-only deployment. An instance is exposed if it has a non-loopback host, a declared TLS terminator, or a public address. An exposed instance disables the default-on console, logs a warning, and serves only JSON. To serve <code>/ui</code> off-box, set <code>serve_web_console = true</code>, configure TLS, and set <code>[security].web_console_public_address</code>. The separately versioned <code>messagefoundry-webconsole</code> wheel mounts in the engine process. It is the sole operator console. The PySide6 desktop console was retired (ADR 0032).
VS Code extension	Visual Studio Code (current stable) — route wizard, validate-on-save, test bench, stage→promote.
Test harness — <i>optional, not needed to run the engine</i>	The synthetic send/receive/load harness (<code>python -m harness</code>) is the only remaining PySide6 (Qt) interface. It has its own distribution, released with the engine, and is not included in the engine wheel. Install it with <code>pip install messagefoundry-harness</code>. This installs <code>messagefoundry[harness]</code>, including PySide6. The harness runs on Windows, Linux, and macOS as a separate HTTP API client. An engine host without the harness needs no Qt or GUI.

Network & ports

Purpose	Default	Notes
Engine API (HTTP + WebSocket)	127.0.0.1:8765	The API requires authentication and binds to loopback by default. In-process TLS is optional (WP-13a, ADR 0002). Set <code>[api].tls_cert_file</code> . If the key is a separate PEM file, also set <code>tls_key_file</code> . Uvicorn then serves the API through <code>https</code> and <code>/ws/stats</code> through <code>wss</code> . The TLS minimum is 1.2. <code>tls_min_version</code> accepts <code>1.2</code> or <code>1.3</code> , and <code>tls_ciphers</code> is optional. Set <code>tls_client_ca_file</code> to require and verify client certificates through mTLS. A TLS-terminating reverse proxy is also supported through <code>tls_terminated_upstream</code> and <code>trusted_proxies</code> . Off-loopback operation requires in-process TLS or a declared proxy, plus the applicable safeguards below. The engine does not check OCSP/CRL revocation. In-process TLS off loopback therefore exits with code 2 unless the environment variable <code>MEFOR_TLS_REVOCATION_ATTESTED=1</code> is set (ADR 0078). This is not a TOML key. At the default enforcement level, a PHI instance behind a proxy also requires <code>proxy_intra_service_auth</code> and <code>proxy_tls_min_version</code> . The browser console rejects an unprotected off-loopback bind.
Inbound MLLP / TCP listeners	operator-defined (samples use <code>2575</code> , <code>2600</code>)	Permit sending systems through the firewall. MLLP-over-TLS is optional per connection through <code>tls = true</code> , with TLS 1.2+ (WP-13b, CONNECTIONS.md). An inbound uses <code>tls_cert_file</code> and <code>tls_key_file</code> for its server identity. Set <code>tls_ca_file</code> to enable mTLS. Outbounds verify the partner certificate and hostname by default: <code>tls_verify</code> and <code>tls_check_hostname</code> are both <code>true</code> . Plaintext remains the default, so a non-loopback MLLP listener requires <code>tls = true</code> . An off-loopback cleartext listener causes a <code>WiringError</code> during wiring. A PHI instance at default enforcement rejects this even with <code>serve --allow-insecure-bind</code> . Default <code>[security].enforcement = enforce</code> also rejects cleartext MLLP egress off loopback, regardless of data class. ADR 0153 removed the synthetic-data exemption. Exceptions require an attested hop, <code>cleartext_accepted = true</code> (warning and audit), or <code>enforcement = warn</code> .
Outbound	as configured	Reachability to downstream partners and, for server DBs, to the database host.
Installer egress	HTTPS	Outbound access for the service installer to fetch the pinned NSSM binary (or pre-stage it).

Sizing by message volume

Use these tiers as sizing starting points. Each peak rate comes from a named measured run listed under *Reading the tiers*. No tier projects above a measured rate. Rates depend on the source hardware, fan-out, message size, strict-validation use, and transform cost. Transform cost per message is the dominant factor. Measure your feeds with the load harness before committing. See [Capacity notes](#). The source rates and conditions are in [benchmarks/TUNING-BASELINE.md](#) and [THROUGHPUT.md](#). The arithmetic below shows how each tier was derived. A tier row is not a deployment guarantee.

How throughput is bounded (read this first)

A single engine process runs **all** message work — decode → peek → route → transform → re-encode — on **one CPU core** (one asyncio event loop, the GIL prevents pure-Python parallelism across threads). So per-process throughput is governed, in order, by:

1. **Transform cost per message** — usually the binding constraint. The project's own [throughput research](#) cites a comparable vendor benchmark where real transformation cut pass-through throughput by ~60% (~1000 msg/s → ~400 msg/s). A light/pass-through feed sits near the top of a tier. A heavy transform sits near the bottom.
2. **Durable-write cost** — every stage handoff (ingress → routed → outbound → delivered) is a committed transaction. In-process **SQLite is fastest per write**. A **server DB is slower per single write** (network + MVCC) but is the concurrency substrate (next point).

Server databases support concurrent workers. Multiple connections, lanes, and delivery workers can drain one PostgreSQL or SQL Server database through `SELECT ... FOR UPDATE SKIP LOCKED` and row leases. Database commit capacity limits throughput. SQLite has one writer and does not scale this way. It is the single-process, single-node store.

Engine HA uses **single-leader active-passive** operation. Only the leader runs the graph. A separate, built multi-process mode partitions inbound Connections through `messagefoundry supervise` (ADR 0037).

More than one engine shard requires **one shared server database** (ADR 0063). [ADR 0073](#) defines ownership-scoped recovery and single-consumer delivery lanes. A restarting engine shard recovers only its own in-flight rows. Deterministic rendezvous ownership assigns each outbound lane to one delivery consumer, which preserves FIFO order.

Startup rejects engine sharding with `[cluster]` active-passive operation. Changes to the engine-shard set require a coordinated restart, and reload rejects them.

Production support remains pending. The mechanism is built and invariant-tested, but multiple active engines on one store are not yet a supported production topology. Support requires a sustained, clean 4-engine benchmark with zero loss and per-lane FIFO. Until then, size production multi-engine deployments as **active-passive**, with one active writer per store.

Connection-count guidance. Before ADR 0066, `per_lane` created one claim loop per connection per stage. Approximately 1,500 idle connections saturated an 8-vCPU store host through lock contention. Default `pooled` mode replaces those loops with a small set of shared `StageDispatcher` claimers (ADR 0066, default since #744). Keep `pooled` mode for high connection counts. With `[pipeline].claim_mode = "per_lane"`, use no more than a few hundred connections per store. Enable `[pipeline].per_lane_wake` to reduce idle claim work to approximately zero.

Limits: Exactly-once behavior degrades under load because inbound duplicate detection is absent. Receivers must be idempotent. The evidence for the mode change covers one node. Failover duplicate and ordering behavior remains unmeasured, including the T17 infrastructure-fault limit tracked by ADR 0070. Details are in the "Pipeline claim mode" section of `CONNECTIONS.md`.

Tiers

Tier	Peak sustained ingress	Indicative daily volume	Deployment shape	Store	Suggested hardware (engine host)
Pilot / light	up to ~30 msg/s	up to ~1.0 M/day	1 process, single node	SQLite	2 cores / 4 GB
Standard single-node	~30–70 msg/s	~1.0–2.2 M/day	1 process, single node	SQLite, or PostgreSQL / SQL Server	4 cores / 8 GB
High single-node	~70–100 msg/s	~2.2–3.2 M/day	1 process, tuned (lean transforms. Low fan-out. The default <code>pooled</code> claim mode. Finite-retry on hot lanes), many connections / lanes draining concurrently via <code>SKIP LOCKED</code>	Server DB on its own host (PostgreSQL / SQL Server)	4–8 cores / 16 GB + a DB host sized to the commit load
Multi-process (engine shards)	~165 msg/s measured at 4 engine shards (per-shard SQLite). Beyond that $\approx N \times$ your measured single-shard rate $\times 0.85$	~5.3 M/day at that measured rate	N <code>messagefoundry supervise</code> engine-shard processes, partitioned by inbound connection — not yet certified as a production topology (see above)	PostgreSQL / SQL Server — required above one shard, so all engine shards share one unified store	8+ cores / 32 GB + a dedicated DB host sized to the commit load

Reading the tiers — and checking the arithmetic

Every peak figure in the table comes from a named measured run. Only the last row's per-shard multiplier is extrapolated. The notes identify hardware combinations that were not measured and show each calculation.

- **Units.** *Peak sustained ingress* is **messages received per second**, at the **low fan-out (~1–2 destinations per received message)** of every anchor run below. A high-fan-out hub commits and delivers several copies per received message and sustains a **much lower** ingress rate — see the last bullet.
- **Daily = peak × 86,400 ÷ 2.7**, i.e. **peak × ~32,000 — not peak × 86,400**. In the production ADT feeds profiled in [THROUGHPUT.md §6](#), the busiest hour was **~2.7× the all-day average**. Sizing must use the peak hour. The daily estimate therefore uses **86,400 / 2.7** seconds at the peak rate. That is the duty cycle assumed in the *indicative daily volume* column, and it is the same arithmetic [THROUGHPUT.md §6](#) works through (60 msg/s → ~1.9 M/day, not ~5.2 M). Substitute your own peak-hour ÷ daily-average ratio once you have measured one.
- **Pilot / light — ~30 msg/s.** This is the **lowest** sustainable rate across the three backends on the reference configuration. The SQL Server run passed conformance checks at **fan-out 2** on a **4-vCPU runner with a co-located database** ([TUNING-BASELINE.md §Results](#)). **No 2-core point was measured.** This tier uses the lowest measured rate. It does not estimate a new rate for the suggested hardware. **30 × 32,000 ≈ 0.96 M/day**.
- **Standard single-node — ~30–70 msg/s** is that same reference config's full measured band: **~30 (SQL Server) · ~50 (PostgreSQL) ≥ 70 (SQLite, still not saturated at the top rate step)**. One strictly ordered interface measured **~60 msg/s end-to-end** against an *instant-acknowledging* partner. Intake reached **~193 msg/s per engine** (ACK-on-receipt, engine-CPU-bound, ~383 msg/s measured at two engines) — i.e. **~16 ms** for the whole serial per-message budget, most of which is store round-trips rather than engine time ([THROUGHPUT.md §8](#)). **30–70 × 32,000 ≈ 0.96–2.24 M/day**. The ~1000 msg/s pass-through figure quoted above is a **vendor's** self-benchmark, not a MEFOR measurement.
- **High single-node — ~70–100 msg/s.** One engine process measured **~97 msg/s sustained**, with zero loss and a bounded backlog. A **~107 msg/s** burst also drained. The test used **1,500 inbound connections**, default **pooled** mode, and fan-out 1. SQL Server 2022 ran on a separate host ([benchmarks/adr0066-pooled-claimer-744.md §2](#)). Engine CPU used ~2 cores throughout. A 3× larger store pool did not raise the rate. This is the measured **single-process limit**. The next tier uses more processes. **70–100 × 32,000 ≈ 2.24–3.2 M/day**.
- **Multi-process (engine shards) — ~165 msg/s at 4 shards.** The measured sequence was 1 → 2 → 4 engine shards at **~50 → 88.7 → 165.5 msg/s** aggregate. Scaling was approximately linear, with efficiency **η ≈ 0.85** per added engine shard ([TUNING-BASELINE.md §Multi-process sharding scale-out](#)). **165.5 × 32,000 ≈ 5.3 M/day**. **Two limits apply.** The run used **per-shard SQLite on a consumer 8-core test host**. The reusable result is the **speedup shape**, rather than the absolute rate: multiply η by *your* measured single-shard rate. And a production multi-shard deployment must share **one unified server-DB store (ADR 0063)** — a shape that run never exercised, and one measured **worse** (next bullet).
- **Where the tiers stop, and why.** Fan-out and the shared store — not the tier label — set the real number. A 900-second soak used **4 engine shards, one shared SQL Server store**, and **fan-out 8**. It sustained **10 msg/s ingress** (80 deliveries/s, 90 total message events/s). A fixed light load scaled only to **4 engine shards**. N = 8 and N = 16 collapsed ([benchmarks/THROUGHPUT-STATUS-2026-07-10.md §3](#)). Per-interface limits do **not add**. A measured 16-lane run delivered **87/s aggregate, or 5.44/s per lane**. Adding the individual limits would predict ~960/s, approximately **11× too high** ([THROUGHPUT.md §7](#)). Always take **min(measured concurrent aggregate, Σ per-interface)**, and measure a high-fan-out hub rather than sizing it off the low-fan-out rows above. **~165 msg/s of ingress is the highest end-to-end rate in the sizing tiers above**, and no figure in this document may be quoted as more. That bound applies to the per-interface ingress rates and hardware in

these tiers. It does not define the engine's aggregate limit. A separate four-shard measurement against a shared SQL Server store reached ~603 total message events per second (counting messages in **and** out). See the Throughput & Capacity document. The two are different units on different hardware and should never be compared directly.

- **Do not size against future transaction reductions.** Group-commit and other reductions in committed transactions per event are **closed, not pending**. Group-commit was withdrawn (ADR 0055). The pre-registered measurement returned ABANDON at elasticity -0.115 (ADR 0107).

Single-stream server-database limit. Each stage handoff commits a database round-trip. The baseline measured approximately **30 msg/s on SQL Server**, compared with **≥70 msg/s on SQLite** for one stream. See [TUNING-BASELINE.md](#), Results. A server database needs concurrent connections, lanes, or processes for high volume. Size its host for that combined commit load.

Capacity notes

- Validate with the load harness ([LOAD-TESTING.md](#)). Run the `smoke` → `fanout-baseline` → `soak` sequence. Test the `cheap`, `edit`, and `slow` transform modes to find the per-core limit. Compare SQLite and a server database with identical traffic. Treat the **zero-loss reconciliation** as the headline gate — throughput is meaningless if messages were lost.
- The embedded store has ~3× write amplification and a single-writer ceiling. Move to PostgreSQL or SQL Server when that becomes the bottleneck.
- Scale **intra-node** on a server DB (one delivery worker per outbound. Many connections / lanes draining concurrently via `SKIP LOCKED`. Keep retry policies finite where head-of-line blocking on a shared FIFO lane would otherwise stall a lane). A multi-process **engine-shard** scale-out beyond one engine **is built** (`messagefoundry supervise` — ADR 0037, ADR 0063, ADR 0073). All engine shards must share one server database. This is **not yet certified as a production topology**. See [Sizing by message volume](#). Engine **HA** is **active-passive failover** (opt-in leader/standby cluster on shared PostgreSQL — see [CLUSTERING.md](#)). Delegate **DB-tier** HA to the database + a load-balancer VIP.